



SMART CONTRACT SECURITY REVIEW

# ICFT Protocol

Security review of the ICFT lending protocol: pool accounting and the index-based debt model, the oracle layer, the liquidation mechanism and administrator powers.

0

● CRITICAL

4

● HIGH

5

● MEDIUM

7

● LOW

3

● INFORMATIONAL

# Document Control

|                |                                                                         |
|----------------|-------------------------------------------------------------------------|
| Client         | ICFT Protocol                                                           |
| Protocol       | ICFT Protocol                                                           |
| Report type    | Smart contract security review                                          |
| Classification | Public release                                                          |
| Auditor        | WhyUSayMyName                                                           |
| Contact        | <a href="https://github.com/WhyUSayMyName">github.com/WhyUSayMyName</a> |

## Version History

| VERSION | DATE        | DESCRIPTION                                                          |
|---------|-------------|----------------------------------------------------------------------|
| v1.0    | 12 Sep 2026 | Initial report delivered to the client. No fix review was performed. |

## About the Auditor

WhyUSayMyName, independent blockchain security researcher. I work across lending markets, vaults, oracle layers, DEXes, stablecoins, governance, cross-chain messaging and bridges.

My process combines line by line manual review, stateful invariant testing and economic scenario modelling. Before hunting for vulnerabilities I build a map of the protocol: which contract calls which, which functions mutate state, where value is held and how it moves, which roles influence what, and which invariants must hold at all times. The map exposes the trust boundaries and the nodes where a mistake costs the most, and that is where the attention goes next.

Every finding must be backed by an executable proof, not only by an argument.

## Use of This Report

This report describes the state of the code at the commit listed in the Scope section. It was delivered to the client to improve the security of the reviewed system and is not an endorsement of the project, the team, the token or the business model. A security review is time boxed and does not guarantee the absence of vulnerabilities. The full limitation of liability is given in Appendix C.

This report is published with the client's consent. At the time of publication the protocol is not deployed to mainnet, and the team is addressing the findings listed here before deployment. The `open` status on a finding reflects the state as of the review date, not the presence of a vulnerability in a future release.

# Contents

|        |                                                                                                                   |    |
|--------|-------------------------------------------------------------------------------------------------------------------|----|
| 01     | Executive Summary                                                                                                 | 05 |
| 02     | Scope                                                                                                             | 08 |
| 03     | Methodology                                                                                                       | 10 |
| 04     | System Overview                                                                                                   | 12 |
| 05     | Trust Model & Privileged Roles                                                                                    | 14 |
| 06     | Risk Classification                                                                                               | 16 |
| 07     | Codebase Maturity                                                                                                 | 17 |
| 08     | Findings Summary                                                                                                  | 18 |
| 09     | Detailed Findings                                                                                                 | 20 |
| ● H-01 | The ICFT price is set by a single role instantly and without limits, which allows value to be drained from Fund A | 20 |
| ● H-02 | Uncovered debt survives after all collateral is seized, and nothing can write it off                              | 22 |
| ● H-03 | A liquidator bonus that passes validation disables liquidation entirely                                           | 24 |
| ● H-04 | Any contract implementation can be replaced by one key in the same block, with no delay                           | 26 |
| ● M-01 | The borrowed token counter is reduced by tokens returned, so utilisation is understated when the ICFT price falls | 28 |
| ● M-02 | During an emergency freeze collateral is locked while interest keeps accruing                                     | 30 |
| ● M-03 | Protocol revenue accumulates in the contract with no way to use it                                                | 32 |
| ● M-04 | No minimum borrow size is enforced, which allows positions that are uneconomical to liquidate                     | 34 |
| ● M-05 | Administrative role handover is a single step and a pause has no time limit                                       | 35 |
| ● L-01 | The Chainlink answer is not fully validated: the round id is ignored and there is no staleness headroom           | 36 |
| ● L-02 | Risk parameter validation allows zero headroom between the target LTV and the liquidation threshold               | 37 |
| ● L-03 | The project build is not reproducible: dependency paths conflict and the compiler version is not pinned           | 38 |
| ● L-04 | Collateral valuation makes an external call per asset while looping over the whole registry                       | 39 |
| ● L-05 | Upgradeable contracts reserve no storage slots                                                                    | 40 |

|          |                                        |                                                                                                              |           |
|----------|----------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------|
|          | ● L-06                                 | Collateral is credited by the requested amount rather than the amount actually received                      | 41        |
|          | ● L-07                                 | The stepped rate curve jumps at segment boundaries and has no upper bound                                    | 42        |
|          | ● I-01                                 | Floating compiler version and incomplete documentation of the public interface                               | 43        |
|          | ● I-02                                 | The external whitepaper describes the protocol differently from the MVP specification and the implementation | 44        |
|          | ● I-03                                 | Interest compounds, and the accrual mode is not fixed in the specification                                   | 46        |
| <b>A</b> | <b>Invariants &amp; Property Tests</b> |                                                                                                              | <b>47</b> |
| <b>B</b> | <b>Automated Analysis Triage</b>       |                                                                                                              | <b>48</b> |
| <b>C</b> | <b>Disclaimer</b>                      |                                                                                                              | <b>49</b> |

# Executive Summary

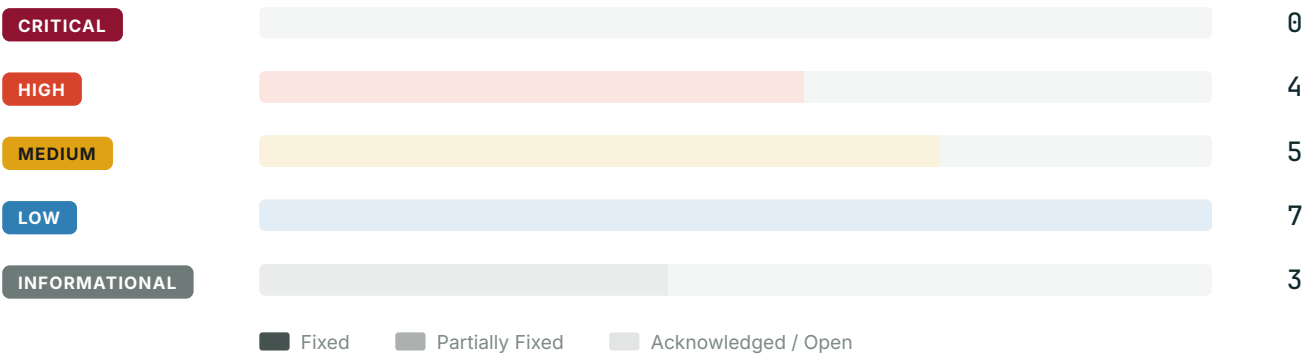
ICFT is a lending protocol with a single borrowing market. A user deposits ETH or a whitelisted collateral token, borrows ICFT and takes on debt denominated in USD. Interest accrues through a global borrow index, liquidation is handled by a separate contract behind an operator role, and prices come from Chainlink for ETH and from a manual source for ICFT.

The review produced **nineteen issues**: four of high severity, five medium, seven low and three informational. No critical issues were found, because every path to a loss of funds requires a privileged key, and the centralised control model is recorded in the specification as a deliberate state of the MVP.

Most findings cluster around two themes: administrator powers that are bounded neither by a delay nor by value limits, and the absence of an insolvency model, which leaves uncovered debt in the system permanently.

|                                                  |                                                                                   |                               |
|--------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------|
| ENGAGEMENT<br>Private review, initial engagement | REVIEW PERIOD<br>1 to 12 September 2026                                           | EFFORT<br>12 days · 1 auditor |
| NSLOC IN SCOPE<br>1,403                          | METHODS<br>Manual review, static analysis, Foundry invariants, economic scenarios | FINDINGS<br>19 · 0 fixed      |

## Findings Overview



**OVERALL ASSESSMENT**

The accounting core is carefully built. Debt is stored in scaled units and grows through a global index, so elapsed periods are not repriced after the fact by a later rate. Rounding direction favours the protocol in most places, and conversion between the decimals of ETH, ICFT, USDT and the oracle answer is symmetric and states its rounding direction explicitly. The code contains no dangerous constructs: no `tx.origin`, no `delegatecall`, no `selfdestruct` and no `unchecked` blocks.

The weak points sit at the edges of the system. The ICFT price comes from a single source that one role sets by hand, with no delay, no step limit and no cross check against a market. That is enough to drain value from the shared fund in two transactions, which is demonstrated by an executable proof. The liquidation mechanism handles healthy scenarios correctly, but it does not model the state in which debt exceeds collateral: the collateral is seized in full, the remaining debt stays in the system, and nothing can write it off.

The validation of risk parameters deserves separate mention. It accepts values that look reasonable yet disable liquidation entirely through an arithmetic overflow. This is not theft of funds, it is the failure of a protective mechanism at the exact moment it is needed.

**Key Observations**

- 01 **The ICFT price rests on trust in a single key.** There is one price source, no second source to cross check against, no deviation limiter and no value bounds. The role that sets the price applies it immediately. The specification allows a manual price for the MVP, but that does not remove the need for limiters.
- 02 **Insolvency is not modelled.** When collateral collapses, liquidation takes all of it and closes only part of the debt. The remainder continues to exist in the accounting, with no write off and no insurance mechanism.
- 03 **A reasonable parameter value switches liquidation off.** A liquidator bonus above roughly 17.6 percent at a target LTV of 85 percent passes validation, after which the liquidation calculation overflows and reverts. Liquidation stops working for the whole protocol.
- 04 **Token accounting drifts away from debt as the price moves.** The counter of borrowed ICFT is reduced by the number of tokens returned rather than by the number originally issued. When the ICFT price falls, one borrower repaying zeroes the counter even though other borrowers still owe, and utilisation is understated for everyone.
- 05 **Tests existed, an invariant over fund accounting did not.** The repository already contains unit, fuzz and invariant tests, but no invariant checked the conservation of fund tokens. That is precisely why the accounting drift never surfaced. The invariant set in Appendix A closes that gap.

**Priorities Before Deployment**

- 01 Close **H-01**: add a delay, a step limit and multi party approval to the functions that set the price and the price source.
- 02 Close **H-02**: define the behaviour when collateral falls below debt and introduce accounting for uncovered debt.
- 03 Close **H-03**: bound the liquidator bonus so that the liquidation calculation cannot overflow.

- 04 Add the invariants from Appendix A to continuous integration and run them before every release.

02

# Scope

The review covered the core protocol contracts at the commit listed below. No fix review is included in this version of the report.

|            |                                     |
|------------|-------------------------------------|
| Repository | ICFT-Protocol                       |
| Commit     | 7135745                             |
| Language   | Solidity                            |
| Compiler   | 0.8.35 (via-IR, optimizer 200 runs) |
| Framework  | Foundry                             |

## Files in Scope

| FILE                                        | NSLOC        | SHA-256 (PREFIX) |
|---------------------------------------------|--------------|------------------|
| src/core/ICFT/lending/LendingPool.sol       | 593          | aa0630616182c9ce |
| src/core/ICFT/oracle/PriceOracle.sol        | 201          | d7504f6aa940b932 |
| src/core/ICFT/lending/LiquidationEngine.sol | 200          | b8699b6de11f408c |
| src/core/ICFT/risk/RiskEngine.sol           | 128          | 3d2a48ab876c7df5 |
| src/core/ICFT/lending/InterestRateModel.sol | 47           | 9b54fd492361405f |
| src/core/ICFT/token/ICFT.sol                | 45           | 3af482a4c65c488a |
| src/core/interfaces/ILendingPool.sol        | 62           | 0e4aa8660ddc8c55 |
| src/core/interfaces/ILiquidationEngine.sol  | 36           | 96cfacbaa97fd355 |
| src/core/interfaces/IRiskEngine.sol         | 23           | 78e011d289d14c0f |
| src/core/interfaces/IPriceOracle.sol        | 21           | 7e912ce42e02bb38 |
| src/core/interfaces/IInterestRateModel.sol  | 16           | 970550165e83faea |
| src/core/utils/Errors.sol                   | 26           | bf3758c52264f8bd |
| src/core/utils/PriceSource.sol              | 5            | a293a25df7081647 |
| <b>Total</b>                                | <b>1,403</b> |                  |

## Out of Scope

- Deployment scripts, tests and mocks ( `script/` , `test/` , `src/mocks/` ).

- Third party dependencies: OpenZeppelin 5.x and the Chainlink interfaces.
- Off chain liquidation bot infrastructure, monitoring and the frontend.
- A protocol owned DEX, swap routing and governance mechanics, all placed outside the MVP by the specification itself.
- Vesting, the Fund B reserve and a minimum borrow size, none of which appear in the MVP specification.

03

# Methodology

The work ran in stages. The threat model built at the start decided where manual review and fuzzing were concentrated, and each stage built on the result of the previous one.

## Review Phases

01

### Reconnaissance and modelling

Documentation review, a dependency map of the contracts, a value flow diagram, and an inventory of roles and of the entry points that move funds.

02

### Static analysis

Slither and solhint across the whole codebase, with every hit triaged by hand (Appendix B).

03

### Manual review by module

Line by line review of the token, the pool, the rate model, the oracle, liquidation, the funds, access control and the emergency freeze.

04

### Invariants and fuzzing

Stateful Foundry handlers that also move prices; the properties are listed in Appendix A.

05

### Proofs

For every finding of medium severity and above an executable test reproduces the behaviour with concrete numbers.

06

### Economic scenarios

Collateral crashes, liquidation profitability against gas costs, ICFT pool depth, bot downtime and sell pressure.

07

### Finding validation

An adversarial second pass over every finding against the source and against the MVP specification. Some findings were corrected, others were reclassified as intended behaviour.

## Tooling

| TOOL           | PURPOSE                                                                                                   |
|----------------|-----------------------------------------------------------------------------------------------------------|
| Foundry        | Proofs, stateful invariant testing, differential comparison against a reference model.                    |
| Slither 0.11.5 | Static analysis, 101 detectors across the codebase.                                                       |
| solhint 6.2.4  | Style, documentation and gas practice checks.                                                             |
| Python         | Reference model of the LTV, interest and partial liquidation formulas, plus numerical economic scenarios. |

## Limitations

The engagement ran for twelve days with one auditor. A fix review is not part of this version of the report. The economic scenarios rest on assumptions about ICFT pool depth, gas price and position distribution: those parameters live in the model and can be varied, and real values at launch may differ.

The behaviour of collateral tokens that deviate from the ERC-20 standard (transfer fees, rebasing, callbacks) was assessed from the code only, without integration runs.

04

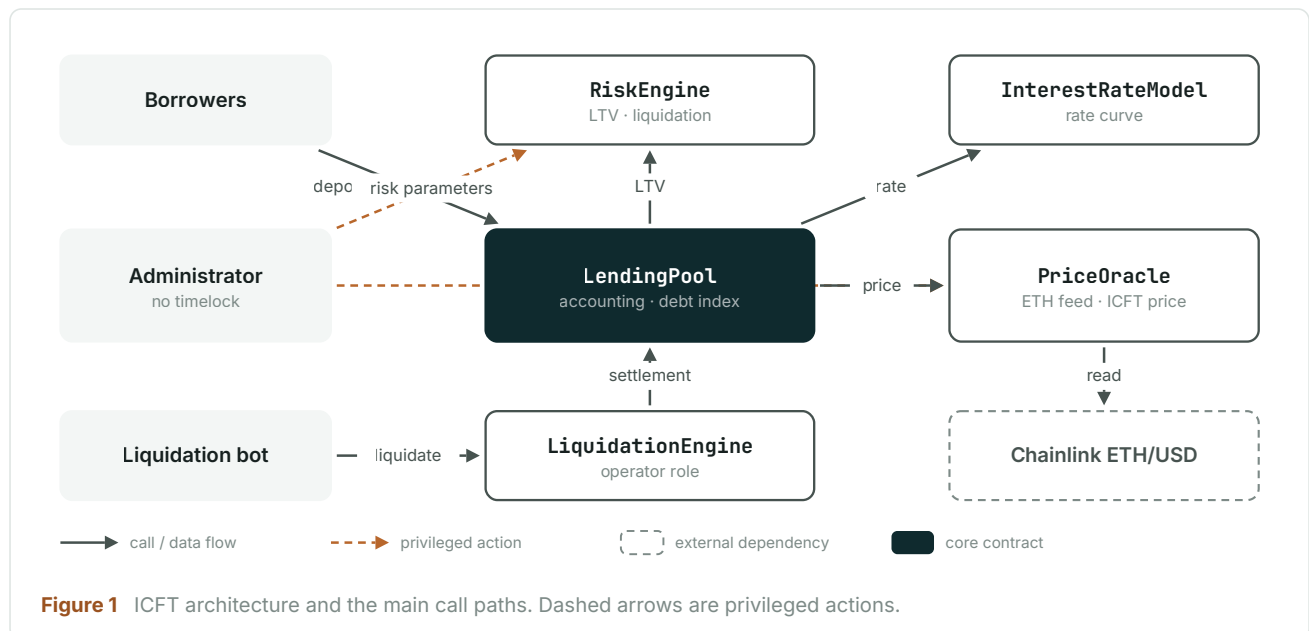
# System Overview

ICFT is a lending protocol with a single borrowing market. A user deposits ETH or a whitelisted collateral token and borrows the ICFT token. Debt is fixed in USD at the moment of borrowing, so the number of tokens needed to repay moves with the ICFT price. That behaviour is described directly in the MVP specification.

Interest accrues through a global borrow index: on every state changing action the index grows by the rate for the elapsed period, and a position's debt is the scaled amount multiplied by the current index.

`InterestRateModel` returns the rate from a stepped utilisation curve, `RiskEngine` holds the risk boundaries, and `PriceOracle` supplies prices.

Liquidation is not open to anyone: `LendingPool.liquidate` is gated by a bot role and `LiquidationEngine.executeLiquidation` by an operator role. The MVP specification permits this arrangement explicitly. All contracts are deployed behind transparent proxies.



## Components

|                          |                                                                                                                                                                   |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>LendingPool</b>       | Holds collateral and the Fund A inventory, tracks scaled debt and the borrow index, checks LTV on borrow and withdrawal, and executes the liquidation settlement. |
| <b>ICFT</b>              | An ERC-20 with a fixed supply of one billion. The entire supply is minted once during initialisation and there is no further mint function.                       |
| <b>PriceOracle</b>       | Normalises prices to 1e18. ETH comes from Chainlink with a staleness check, while the ICFT price is either set manually or read from a feed.                      |
| <b>RiskEngine</b>        | Computes LTV, decides whether a position may be liquidated and how much to close to reach the target LTV, and stores the risk parameters.                         |
| <b>InterestRateModel</b> | A four segment stepped curve that returns the annual rate for a given utilisation and the utilisation ceiling for new borrows.                                    |
| <b>LiquidationEngine</b> | A wrapper around pool liquidation with a preview. It pulls ICFT from the operator, sends collateral to the beneficiary and returns the unused remainder.          |

05

# Trust Model & Privileged Roles

The protocol relies on a set of privileged roles. By default all of them, together with the right to upgrade, land on the single deployer address unless environment variables say otherwise. The table lists what each role can do and what its compromise would mean.

| ROLE                             | HELD BY                              | CAPABILITIES                                                                                 | IF COMPROMISED                                                                                                                 |
|----------------------------------|--------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Proxy owner</b>               | One address, the deployer by default | Replaces the implementation of any of the six contracts through <code>ProxyAdmin</code> .    | Full control over the protocol and the funds. The upgrade executes in the same block, leaving users no window to react (H-04). |
| <b>ORACLE_ADMIN</b>              | One address, the deployer by default | Sets the ICFT price and the price source, changes feed addresses and the maximum answer age. | Arbitrary valuation of obligations. A demonstrated path to draining value from Fund A (H-01).                                  |
| <b>RISK_ADMIN</b>                | One address, the deployer by default | Sets the maximum LTV, the liquidation threshold, the target LTV and the liquidator bonus.    | A bonus value inside the permitted range disables liquidation entirely (H-03).                                                 |
| <b>RATE_ADMIN</b>                | One address, the deployer by default | Sets the rate curve with no upper bound.                                                     | An arbitrarily high rate applied to loans that are already open.                                                               |
| <b>CONFIG_ADMIN</b>              | One address, the deployer by default | Manages the liquidity buffer and the list of collateral assets.                              | Adding an asset with an incorrect feed distorts collateral valuation.                                                          |
| <b>PAUSER</b>                    | One address, the deployer by default | Halts deposits, borrowing, withdrawals and liquidation.                                      | Collateral is locked, interest keeps accruing, and unpausing has no time limit (M-02, M-05).                                   |
| <b>LIQUIDATION_BOT, OPERATOR</b> | Team bot                             | The only parties able to start a liquidation.                                                | While the bot is down unhealthy positions stay open, and there is no external pool of liquidators to fall back on.             |

## Trust Assumptions

- The role keys are controlled by the team and are not compromised. Neither multi signature approval nor a delay exists in the code or in the default settings.
- The manual ICFT price reflects the real market. Nothing on chain verifies this: there is no second source and no deviation limiter.
- The liquidation bot runs continuously and holds enough ICFT to close positions during market stress.
- Collateral assets behave as standard ERC-20 tokens, without transfer fees, rebasing or callbacks.

- The MVP specification is treated as the governing document. Divergences from the external whitepaper are covered in finding I-02.

# Risk Classification

Severity follows from the combination of impact on the system and the likelihood of exploitation. Status reflects the state of the work on an issue at the time this report was issued.

|        |        | LIKELIHOOD |        |        |
|--------|--------|------------|--------|--------|
|        |        | HIGH       | MEDIUM | LOW    |
| IMPACT | HIGH   | CRITICAL   | HIGH   | MEDIUM |
|        | MEDIUM | HIGH       | MEDIUM | LOW    |
|        | LOW    | MEDIUM     | LOW    | LOW    |

Severity is derived from impact and likelihood.

## Severity Levels

|               |                                                                                                                                  |
|---------------|----------------------------------------------------------------------------------------------------------------------------------|
| CRITICAL      | Direct and likely loss of user or protocol funds, or irreversible insolvency. Fix before deployment.                             |
| HIGH          | Loss of funds or failure of a protective mechanism under realistic though limited conditions. Should be fixed before deployment. |
| MEDIUM        | Limited loss, loss under unlikely conditions, or a material deviation from the stated behaviour.                                 |
| LOW           | A minor deviation from the specification or from common practice with little direct impact.                                      |
| INFORMATIONAL | Code quality, readability and defence in depth. No direct security impact.                                                       |

## Resolution Status

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| FIXED           | The issue is resolved and the fix has been verified.            |
| PARTIALLY FIXED | The fix reduces the risk without removing it entirely.          |
| ACKNOWLEDGED    | The team accepts the risk and does not plan to change the code. |
| OPEN            | No fix had been provided at the time this report was issued.    |

07

# Codebase Maturity

This assessment describes engineering practice as a whole, beyond the individual findings, and shows which areas need attention first.

| AREA                  | RATING                                                                                            | SUMMARY                                                                                                                                                                            |
|-----------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Arithmetic            | <br>Satisfactory | A single normalisation to 1e18, an explicit rounding direction at each conversion, and debt rounded up in the protocol's favour. There are no <code>unchecked</code> blocks.       |
| Access control        |  Weak            | Roles are separated in code but converge on one address by default. There is no delay and no multi signature approval, and role handover is a single step (H-04, M-05).            |
| Complexity management | <br>Satisfactory | Contracts are split along clear responsibilities and the liquidation calculation lives in its own module.                                                                          |
| Decentralization      |  Missing         | Every power and the right to upgrade sit with one key, and the upgrade executes instantly. The specification places the final proxy design outside the MVP.                        |
| Documentation         |  Moderate      | The MVP specification is detailed and matches the code on the key values. In the code itself there are 250 NatSpec issues and part of the public interface is undocumented (I-01). |
| Events and monitoring |  Moderate      | Events cover every operation that moves funds, but key fields are not indexed, which makes off chain observation harder.                                                           |
| Insolvency handling   |  Missing       | The state in which debt exceeds collateral is not modelled. The uncovered remainder stays in the accounting with no write off mechanism (H-02).                                    |
| Low level code        |  Strong        | No <code>tx.origin</code> , no <code>delegatecall</code> and no <code>selfdestruct</code> within scope. The two low level calls exist to send ETH and both check the result.       |
| Testing               |  Moderate      | Unit, fuzz and invariant tests all exist, but no invariant covered the conservation of fund tokens, which is why the accounting drift went unnoticed (M-01).                       |

08

# Findings Summary

| ID   | TITLE                                                                                                             | SEVERITY | STATUS |
|------|-------------------------------------------------------------------------------------------------------------------|----------|--------|
| H-01 | The ICFT price is set by a single role instantly and without limits, which allows value to be drained from Fund A | HIGH     | OPEN   |
| H-02 | Uncovered debt survives after all collateral is seized, and nothing can write it off                              | HIGH     | OPEN   |
| H-03 | A liquidator bonus that passes validation disables liquidation entirely                                           | HIGH     | OPEN   |
| H-04 | Any contract implementation can be replaced by one key in the same block, with no delay                           | HIGH     | OPEN   |
| M-01 | The borrowed token counter is reduced by tokens returned, so utilisation is understated when the ICFT price falls | MEDIUM   | OPEN   |
| M-02 | During an emergency freeze collateral is locked while interest keeps accruing                                     | MEDIUM   | OPEN   |
| M-03 | Protocol revenue accumulates in the contract with no way to use it                                                | MEDIUM   | OPEN   |
| M-04 | No minimum borrow size is enforced, which allows positions that are uneconomical to liquidate                     | MEDIUM   | OPEN   |
| M-05 | Administrative role handover is a single step and a pause has no time limit                                       | MEDIUM   | OPEN   |
| L-01 | The Chainlink answer is not fully validated: the round id is ignored and there is no staleness headroom           | LOW      | OPEN   |
| L-02 | Risk parameter validation allows zero headroom between the target LTV and the liquidation threshold               | LOW      | OPEN   |
| L-03 | The project build is not reproducible: dependency paths conflict and the compiler version is not pinned           | LOW      | OPEN   |
| L-04 | Collateral valuation makes an external call per asset while looping over the whole registry                       | LOW      | OPEN   |
| L-05 | Upgradeable contracts reserve no storage slots                                                                    | LOW      | OPEN   |
| L-06 | Collateral is credited by the requested amount rather than the amount actually received                           | LOW      | OPEN   |
| L-07 | The stepped rate curve jumps at segment boundaries and has no upper bound                                         | LOW      | OPEN   |

| ID   | TITLE                                                                                                        | SEVERITY      | STATUS |
|------|--------------------------------------------------------------------------------------------------------------|---------------|--------|
| I-01 | Floating compiler version and incomplete documentation of the public interface                               | INFORMATIONAL | OPEN   |
| I-02 | The external whitepaper describes the protocol differently from the MVP specification and the implementation | INFORMATIONAL | OPEN   |
| I-03 | Interest compounds, and the accrual mode is not fixed in the specification                                   | INFORMATIONAL | OPEN   |

09

# Detailed Findings

H-01

HIGH

OPEN

**The ICFT price is set by a single role instantly and without limits, which allows value to be drained from Fund A**

## IMPACT

High

## LIKELIHOOD

Medium

## CATEGORY

Oracle layer

## LOCATION

src/core/ICFT/oracle/PriceOracle.sol#L243-L245  
 src/core/ICFT/oracle/PriceOracle.sol#L288-L296  
 src/core/ICFT/oracle/PriceOracle.sol#L198-L204

## DESCRIPTION

The ICFT price comes from a single source. The `icftPriceSource` field defaults to `Manual`, in which case `getICFTUSDPrice()` returns whatever value the administrator wrote by hand. `_setManualICFTPrice()` performs exactly two checks: the price is not zero and the decimals do not exceed eighteen.

src/core/ICFT/oracle/PriceOracle.sol

solidity

```
288 function _setManualICFTPrice(uint256 price, uint8 decimals_) internal {
289     if (price == 0) revert InvalidManualPrice();
290     if (decimals_ > 18) revert UnsupportedPriceDecimals();
291
292     manualICFTPrice = price;
293     manualICFTPriceDecimals = decimals_;
```

There is no delay between proposing a price and applying it, no limit on the size of a single move, no cross check against a market source and no bounds on the acceptable value. The functions `setManualICFTPrice`, `setICFTPriceSource`, `setNativeUSDFeed`, `setICFTUSDFeed`, `setCollateralAssetFeed` and `setMaxPriceAge` all sit behind the same `ORACLE_ADMIN_ROLE` and take effect in the same block.

The MVP specification permits a manual price as a temporary arrangement, but it does not follow that the price may be changed without limiters.

## IMPACT

Debt is denominated in USD, so the ICFT price governs both how many tokens are issued against a given debt and how many tokens are required to close it. That gives two symmetric ways to extract value from the shared liquidity.

Lowering the price before a borrow hands the borrower more tokens for the same USD debt. Once the price returns to its market level, the debt closes with fewer tokens and the difference stays with the borrower. Raising the price before a repayment closes an existing debt with fewer tokens and releases the collateral in full.

Both operations are executed by a single role holder and need no agreement from anyone else. In both cases the money comes out of Fund A liquidity, which is protocol capital.

## PROOF OF CONCEPT

Both scenarios were reproduced against a local deployment of the protocol.

Lowering the price to `$0.50` at borrow time: the same collateral yields `3,200 ICFT` instead of `1,600 ICFT` at a price of `$1`. The debt is booked as `$1,600`. Once the price returns to `$1`, closing it in full requires `1,600 ICFT`, leaving the borrower `1,600 ICFT` of pure profit.

Raising the price to `$2` at repayment time: a debt taken out as `1,600 ICFT` is closed by transferring `800 ICFT`, after which the collateral is withdrawn in full.

```
1 forge test --mt test_S3_manipulateDownAtBorrow_freeTokens
2 forge test --mt test_S4_manipulateUpAtRepay_cheapClose
```

## RECOMMENDATION

Constrain every change of the price and of the price source so that one compromised role cannot reprice the protocol's obligations in a single transaction.

Introduce a delay between proposing a new price and applying it, long enough for borrowers to react. Limit the size of a single move relative to the current value so that a single operation cannot shift the price beyond a sane range. Move the role behind multi party approval.

src/core/ICFT/oracle/PriceOracle.sol

solidity

```
function _setManualICFTPrice(uint256 price, uint8 decimals_) internal {
    if (price == 0) revert InvalidManualPrice();
    if (decimals_ > 18) revert UnsupportedPriceDecimals();
+   uint256 next = normalizePrice(price, decimals_);
+   uint256 prev = manualICFTPrice == 0 ? next : normalizePrice(manualICFTPrice,
    manualICFTPriceDecimals);
+   uint256 delta = next > prev ? next - prev : prev - next;
+   if (delta * BPS > prev * maxPriceDeviationBps) revert PriceDeviationTooLarge();

    manualICFTPrice = price;
    manualICFTPriceDecimals = decimals_;
```

Separately, add a second source for cross checking and a breaker that halts the protocol when the sources diverge beyond a threshold. Before a mainnet launch this should be treated as blocking: for as long as the price rests on trust in one key, every obligation in the protocol rests on it too.

H-02 HIGH OPEN

## Uncovered debt survives after all collateral is seized, and nothing can write it off

## IMPACT

High

## LIKELIHOOD

Medium

## CATEGORY

Liquidation logic

## LOCATION

```
src/core/ICFT/lending/LendingPool.sol#L610-L659  
src/core/ICFT/lending/LendingPool.sol#L577-L608
```

## DESCRIPTION

The settlement calculation in `_calculateLiquidationSettlement()` caps the seizure at the borrower's remaining collateral and reduces the debt to cover down to the amount that this collateral pays for once the bonus is taken into account.

```
src/core/ICFT/lending/LendingPool.sol
```

solidity

```
635 if (seizeAmount > collateralBalance) {  
636     seizeAmount = collateralBalance;  
637 }  
638  
639 uint256 actualSeizedValueUSD = riskEngine.getCollateralValueUSD(collateralAsset, seizeAmount);
```

That behaviour is correct in the narrow sense that the call does not revert and the liquidator receives exactly what exists. The problem lies elsewhere: when the collateral does not cover the whole debt, the difference does not go anywhere. The position is left with a positive `scaledDebtUSD` and zero collateral, and the protocol has no counter of uncovered debt, no write off function and no reserve from which that difference could be paid.

Formally the position still counts as liquidatable, because `calculateLTV` returns the maximum value when collateral is zero. There is simply nothing left to seize, so every further liquidation call is rejected.

## IMPACT

The remaining debt stays in `totalScaledDebtUSD` and `totalPrincipalDebtUSD` forever. It keeps growing with the borrow index even though no collateral and no solvent borrower stand behind it. The protocol's aggregate debt is therefore overstated, and the ICFT lent against that debt will not return to Fund A.

The sharper the move in collateral price, the more such positions appear. A numerical model over a book of a thousand positions skewed toward the maximum LTV shows that a forty percent drop in ETH makes roughly ninety four percent of positions liquidatable, of which about eighty nine percent fall below the value of their collateral. This is no longer an edge case, it is the state the protocol enters under ordinary market stress.

The picture is made worse by liquidation being role gated and requiring the liquidator to hold ICFT. Outside participants cannot step in, and in the same model a bot outage of one day during a twenty five percent collateral drop leaves around eight hundred positions unclosed.

## PROOF OF CONCEPT

A position with one ETH of collateral and roughly \$1,600 of debt reaches an LTV of 160% after ETH falls to \$1,000. Liquidation seizes the collateral down to zero and closes only part of the debt.

The residual debt after liquidation is `$647.62` against zero collateral. Further liquidation attempts are rejected, and the protocol holds nothing that could cover the remainder.

```
1 forge test --mt test_P3_05_badDebtSurvivesFullLiquidation
```

#### RECOMMENDATION

Model insolvency explicitly instead of leaving it as a by-product of the arithmetic.

Introduce separate accounting for uncovered debt. When the seizure is capped by the remaining collateral and part of the debt is left unpaid, write that remainder off the position and move it into a bad debt accumulator, reducing the aggregate totals. The position then closes for good, and aggregate debt stops carrying an amount that is known to be unrecoverable.

The next question is who pays for that accumulator. The usual options apply: a reserve funded from protocol revenue, an insurance fund, or socialising the loss across liquidity providers. The choice belongs to protocol economics, but it has to be made and encoded, otherwise the loss simply goes unrecorded.

It is also worth revisiting the role gate on liquidation, or running backup liquidators, so that positions are closed before they become insolvent rather than after.

H-03

HIGH

OPEN

## A liquidator bonus that passes validation disables liquidation entirely

## IMPACT

High

## LIKELIHOOD

Medium

## CATEGORY

Risk parameters

## LOCATION

```
src/core/ICFT/risk/RiskEngine.sol#L233-L253  
src/core/ICFT/risk/RiskEngine.sol#L180-L184
```

## DESCRIPTION

Validation of the risk parameters checks the liquidator bonus only against an upper bound of one hundred percent.

```
src/core/ICFT/risk/RiskEngine.sol
```

solidity

```
239 bool invalid = maxLtvBps_ == 0  
240     || maxLtvBps_ >= liquidationThresholdBps_  
241     || targetLtvBps_ > liquidationThresholdBps_  
242     || liquidationThresholdBps_ > BPS  
243     || liquidationBonusBps_ > BPS;
```

The liquidation calculation then uses that bonus inside a denominator computed as a subtraction with no protection against crossing zero.

```
src/core/ICFT/risk/RiskEngine.sol
```

solidity

```
180 uint256 denominator = (BPS * BPS) - (targetLtvBps * (BPS + liquidationBonusBps));  
181 if (denominator == 0) revert InvalidRiskParameters();
```

There is a check for zero but none for a negative result, because in unsigned arithmetic no such result exists: the expression simply overflows and the call reverts. The condition for the calculation to work is that the target LTV multiplied by the sum of the base and the bonus must stay below the square of the base. At a target LTV of eighty five percent that puts the bonus ceiling at roughly seventeen and six tenths percent. Validation knows nothing about it.

## IMPACT

A bonus of twenty percent raises no suspicion and could easily be set while tuning parameters to market conditions. The transaction succeeds and nothing signals an error. From that moment every call to `calculateLiquidation` reverts, and with it `liquidate`, `executeLiquidation` and the preview in the liquidation engine all stop working.

The protocol loses the single mechanism that protects it from accumulating bad debt, and it loses it entirely rather than for one position. The failure surfaces not when the parameter is changed but at the first attempt to liquidate, which is to say during market stress. Recovery requires another parameter change, which needs access to the role and takes time.

The mistake is reversible and no funds are lost directly, so this is not critical severity. Still, a protective mechanism failing exactly when it is needed leads to the same outcome as H-02.

## PROOF OF CONCEPT

A bonus of twenty percent is accepted by `setRiskParameters` without error, after which `calculateLiquidation` reverts. The same holds at one hundred percent. At seventeen percent the calculation still runs, which confirms the boundary at roughly seventeen and six tenths percent.

```
1 forge test --mt test_P3_AC3_highBonusBricksLiquidation
```

## RECOMMENDATION

Validate the actual condition under which the calculation stays correct, rather than an abstract ceiling of one hundred percent. That is the only check that genuinely protects against a misconfiguration.

src/core/ICFT/risk/RiskEngine.sol

solidity

```
bool invalid = maxLtvBps_ == 0
    || maxLtvBps_ >= liquidationThresholdBps_
    || targetLtvBps_ > liquidationThresholdBps_
    || liquidationThresholdBps_ > BPS
    || liquidationBonusBps_ > BPS;

+
+ // the liquidation denominator must stay positive
+ if (targetLtvBps_ * (BPS + liquidationBonusBps_) >= BPS * BPS) {
+     revert InvalidRiskParameters();
+ }
```

It is also worth setting an independent sane ceiling on the bonus, on the order of fifteen percent, to rule out values that make no economic sense even when the arithmetic holds.

H-04

HIGH

OPEN

## Any contract implementation can be replaced by one key in the same block, with no delay

## IMPACT

High

## LIKELIHOOD

Low

## CATEGORY

Access control and upgradeability

## LOCATION

```
script/DeployICFTProtocol.s.sol#L75-L80  
script/DeployICFTProtocol.s.sol#L162-L172
```

## DESCRIPTION

All six protocol contracts are deployed behind a `TransparentUpgradeableProxy`. The right to upgrade belongs to the owner of the corresponding `ProxyAdmin`, and a call to `upgradeAndCall` executes immediately. There is no timelock anywhere in the arrangement.

The owner is chosen at deployment time and defaults to the protocol administrator, which in turn defaults to the deployer.

```
script/DeployICFTProtocol.s.sol
```

solidity

```
163 config.admin = vm.envOr("ADMIN_ADDRESS", deployer);  
164 config.upgradeAdmin = vm.envOr("UPGRADE_ADMIN_ADDRESS", config.admin);
```

The recipients of every supply allocation are chosen the same way. If the environment variables are left unset, a single address ends up with every role in every module, the right to upgrade, and the entire billion tokens.

## IMPACT

An upgrade removes every other constraint, because it replaces the logic itself. A new implementation can add minting to a token whose supply is nominally fixed, rewrite debt accounting, or move collateral and Fund A liquidity out of the protocol. None of the checks described in the other findings apply once that happens.

What matters here is not that the power exists, but that it executes instantly. Users have no interval in which they could notice a pending change and close their positions. Any compromise of the key turns immediately into an irreversible loss of funds.

Likelihood is rated low because the scenario requires a compromised or dishonest key holder rather than a flaw in the code. The impact is maximal, which is what produces the high rating.

The MVP specification places the final proxy design outside the current stage, so for a testnet this is an acceptable state. For mainnet it should be treated as blocking.

## PROOF OF CONCEPT

The owner of `ProxyAdmin` replaces the `PriceOracle` implementation with an arbitrary one inside a single transaction. The test confirms that the block timestamp does not move, meaning there is no delay between the decision and its effect.

```
1 forge test --mt test_P3_AC2_instantUpgradeNoTimeLock
```

**RECOMMENDATION**

Transfer ownership of every `ProxyAdmin` instance to a `TimeLockController` with a delay comparable to the time a user needs to close a position. Common practice is somewhere between one and three days.

Make the timelock itself owned by a multi signature wallet rather than a single address. Separately, split the protocol administrator role from the upgrade owner: today they coincide by default, so compromising one key grants both day to day control and the ability to replace the code.

Before a mainnet deployment it is worth adding a guard to the script that refuses to proceed unless the role addresses and supply recipients are set explicitly. The current fallback to the deployer is convenient for local runs but dangerous because it silently produces full centralisation.

M-01 MEDIUM OPEN

## The borrowed token counter is reduced by tokens returned, so utilisation is understated when the ICFT price falls

## IMPACT

Medium

## LIKELIHOOD

Medium

## CATEGORY

Pool accounting

## LOCATION

```
src/core/ICFT/lending/LendingPool.sol#L341-L343
src/core/ICFT/lending/LendingPool.sol#L786-L788
src/core/ICFT/lending/LendingPool.sol#L848-L850
```

## DESCRIPTION

Utilisation is computed as the ratio of borrowed ICFT to the size of Fund A. That metric is prescribed directly by the MVP specification, so the question is not the metric itself but how its numerator is maintained.

On a borrow, `totalBorrowedICFT` grows by the number of tokens issued. On a repayment it is reduced by the number of tokens the borrower returned, using a saturating subtraction.

```
src/core/ICFT/lending/LendingPool.sol
```

solidity

```
341 fundALiquidityICFT += returnedPrincipalICFT;
342 protocolRevenueICFT += returnedRevenueICFT;
343 totalBorrowedICFT = _saturatingSub(totalBorrowedICFT, returnedPrincipalICFT);
```

Debt is denominated in USD, so the number of tokens returned need not match the number issued. When the ICFT price falls, a borrower returns more tokens than they took, and more is subtracted from the shared counter as well. That surplus is charged not against their own loan but against the global counter, which is to say against the other borrowers. The saturation inside `_saturatingSub` hides the underflow and drives the counter to zero.

## IMPACT

Utilisation stops reflecting the real load on the fund. Since the rate depends on it, an understated utilisation moves the curve into its lowest segment and every active borrower pays less than they should. When the price moves the other way the effect reverses: the counter stays above zero after a full repayment and the rate ends up overstated.

No funds are lost directly here; the issue is economic accuracy and the correctness of the accounting. What matters is that the distortion accumulates on every repayment made at a changed price and has no mechanism to correct itself.

Invariant `INV-08` belongs here as well: the sum of fund liquidity and tokens lent out stops equalling the allocated size of the fund.

## PROOF OF CONCEPT

Two borrowers take `1,600 ICFT` each, so the shared counter reads `3,200 ICFT`. The ICFT price then falls ten-fold and the first borrower closes their debt, which requires returning `16,000 ICFT`.

After that `totalBorrowedICFT` is zero even though the second borrower still owes `$1,600`. Utilisation reads zero while debt is outstanding.

```
1 forge test --mt test_F2_07_utilizationDistortedWithTwoBorrowers
```

**RECOMMENDATION**

Reduce the counter not by the number of tokens actually returned but by the share that corresponds to the portion of that borrower's debt being closed. A simple approach is to store the number of tokens originally issued in the position and subtract from the global counter in proportion to the principal repaid, rather than by the tokens received.

The saturation in `_saturatingSub` should be removed at this call site or replaced by an explicit error. As it stands it masks a situation that correct accounting would never produce, and so prevents the divergence from being noticed.

Invariant `INV-08` from Appendix A should go into continuous integration: its absence from the existing test set is exactly why this divergence went unnoticed.

M-02

MEDIUM

OPEN

## During an emergency freeze collateral is locked while interest keeps accruing

**IMPACT**

Medium

**LIKELIHOOD**

Medium

**CATEGORY**

Emergency freeze

**LOCATION**

```
src/core/ICFT/lending/LendingPool.sol#L268-L275  
src/core/ICFT/lending/LendingPool.sol#L420-L422
```

**DESCRIPTION**

A pause blocks deposits, borrowing, collateral withdrawal and liquidation. Repayment stays available, which is a deliberate decision recorded in the MVP specification.

Interest accrual is not covered by the pause. The `accrueInterest()` function is public and carries no `whenNotPaused` modifier, and `repay()` synchronises the position and advances the global index in any case.

```
src/core/ICFT/lending/LendingPool.sol
```

solidity

```
420 function accrueInterest() external {  
421     _accrueGlobalInterest();  
422 }
```

The result is a combination in which debt keeps growing while collateral cannot be withdrawn. A borrower can only put value into the system, never take it out.

**IMPACT**

A borrower who repays their debt in full during a pause still cannot retrieve their collateral until it is lifted. A borrower who cannot repay watches the debt grow with no way out. Liquidation is disabled at the same time, so positions that fall below the threshold stay open and deteriorate, and once the pause lifts some of them will already be insolvent, which compounds H-02.

It is worth noting separately that the specification says nothing about accrual during a pause. This is therefore not a breach of stated behaviour but an unresolved question which, combined with the block on withdrawals, works against the user.

**PROOF OF CONCEPT**

A position with five ETH of collateral and `$5,000.00` of debt at the moment the pause begins. Thirty days later the debt stands at `$5,020.55`. An attempt to withdraw collateral reverts, while repayment goes through.

```
1 forge test --mt test_P3_AC4_freezeLocksCollateralWhileInterestAccrues
```

**RECOMMENDATION**

Decide what the pause is for and make the behaviour match.

If the pause protects the protocol from ongoing damage, it is reasonable to stop interest accrual for its duration so that users do not pay for a period in which they cannot manage their positions. Technically it is enough not to advance `lastAccrualTime` across the paused interval.

If accrual is meant to continue, then collateral withdrawal should be allowed for positions that carry no debt: such a withdrawal creates no risk for the protocol and releases the user from the trap. Either way, the chosen behaviour needs to be written into the documentation rather than left as an implicit consequence of which modifiers were applied.

The block on liquidation during a pause deserves a second look as well, since the freeze currently halts the one mechanism that reduces bad debt.

M-03

MEDIUM

OPEN

## Protocol revenue accumulates in the contract with no way to use it

**IMPACT**

Medium

**LIKELIHOOD**

High

**CATEGORY**

Pool accounting

**LOCATION**

```
src/core/ICFT/lending/LendingPool.sol#L172
src/core/ICFT/lending/LendingPool.sol#L341-L342
src/core/ICFT/lending/LendingPool.sol#L502-L505
```

**DESCRIPTION**

The interest portion of every repayment is separated from principal and recorded in `protocolRevenueICFT`. The same amount is then excluded from the inventory available for new loans.

```
src/core/ICFT/lending/LendingPool.sol
```

solidity

```
502 function getSpendablePrincipalBalance() public view returns (uint256 spendablePrincipalICFT) {
503     uint256 rawBalance = icft.balanceOf(address(this));
504     return _saturatingSub(rawBalance, protocolRevenueICFT);
505 }
```

The variable appears five times in the contract: the declaration, two increments on repayment and liquidation, and two reads. No function reduces it, withdraws the accumulated amount or distributes it to anyone.

**IMPACT**

The ICFT corresponding to accrued revenue stays on the contract balance while being excluded from available liquidity. It can neither be lent out nor taken out. In effect those tokens are removed from circulation without any explicit intent to burn them.

The amount grows continuously with repayments. Neither liquidity providers nor a protocol treasury can reach that revenue, which means the economic model in which interest accrues to someone is not implemented in code. The only way to reach these funds today is an implementation upgrade, which brings us back to H-04.

Likelihood is high because this is not an attack scenario but the ordinary course of operation: the first repayment that carries interest is enough.

**PROOF OF CONCEPT**

A loan of ten thousand ICFT, a year of accrual, then a full repayment. Afterwards `protocolRevenueICFT` stands at `500 ICFT`, and the inventory available for new loans is reduced by exactly that amount. A search through the sources confirms that no function ever decreases the variable.

```
1 forge test --mt test_P3_FUND2_revenueAccumulatesAndIsLocked
```

**RECOMMENDATION**

Add a function that withdraws accrued revenue to a treasury address behind a dedicated role, with an event and a check that the amount withdrawn does not exceed the recorded balance.

If the intent is instead that revenue should flow back into Fund A and increase available liquidity, that too needs to be expressed in code rather than left as a consequence of the tokens merely sitting on the balance. As it stands the behaviour matches neither sensible option: the funds are neither distributed nor put to work.

Whichever scheme is chosen should also be recorded in the specification, since it is described neither there nor in the code today.

M-04

MEDIUM

OPEN

## No minimum borrow size is enforced, which allows positions that are uneconomical to liquidate

**IMPACT**

Medium

**LIKELIHOOD**

Medium

**CATEGORY**

Pool accounting

**LOCATION**`src/core/ICFT/lending/LendingPool.sol#L277-L279`**DESCRIPTION**

The only check on the size of a loan is that it differs from zero.

`src/core/ICFT/lending/LendingPool.sol`

solidity

```
277 function borrow(uint256 amountICFT) external nonReentrant whenNotPaused {
278     if (amountICFT == 0) revert ZeroAmount();
```

There is no lower bound in value terms, so a position can be opened for a single indivisible unit of ICFT. An upper bound does exist: both the LTV and the utilisation ceiling are checked.

**IMPACT**

A liquidation earns the liquidator a bonus of five percent of the debt covered and costs them gas. That sets a debt size below which closing a position makes no economic sense. At a gas price of two hundred gwei and ETH around \$1,600 the break even point sits at roughly \$1,920, and at four hundred gwei it doubles.

Positions below that threshold will not be closed by anyone, because the liquidator would lose more on gas than they gain. They stay in the system and, on an adverse price move, turn into the uncovered debt described in H-02.

On its own this is a slow leak rather than a single loss. It can be accelerated deliberately, though: creating a large number of small positions costs the attacker only gas while leaving the protocol with debt that cannot be closed at a profit. No extra load is placed on any loop in the contract, since positions are never iterated, so there is no denial of service here. The harm is purely economic.

**PROOF OF CONCEPT**

A loan of one indivisible unit of ICFT succeeds and creates a position with non zero scaled debt.

```
1 forge test --mt test_P3_03_dustBorrowAccepted
```

**RECOMMENDATION**

Introduce a minimum borrow size in value terms and check it inside `borrow()` after converting the amount to USD. The value is worth making configurable, since the break even point depends on the gas price and on the network the protocol runs in.

It is also worth enforcing a minimum remaining position size after a partial liquidation, so that closing a position does not leave behind a remainder that is itself below the profitability threshold.

M-05

MEDIUM

OPEN

## Administrative role handover is a single step and a pause has no time limit

**IMPACT**

Medium

**LIKELIHOOD**

Low

**CATEGORY**

Access control and upgradeability

**LOCATION**

```
src/core/ICFT/lending/LendingPool.sol#L399-L405  
src/core/ICFT/lending/LendingPool.sol#L204-L211
```

**DESCRIPTION**

Permissions are built on `AccessControlUpgradeable`. Roles are granted and revoked in a single call, with no confirmation required from the receiving address. The `AccessControlDefaultAdminRules` variant, which adds a two step handover and a delay, is not used.

The pause is a separate matter. Only the holder of `PAUSER_ROLE` can lift it, and there is no automatic expiry.

```
src/core/ICFT/lending/LendingPool.sol
```

solidity

```
399 function pause() external onlyRole(PAUSER_ROLE) {  
400     _pause();  
401 }  
402  
403 function unpause() external onlyRole(PAUSER_ROLE) {  
404     _unpause();  
405 }
```

**IMPACT**

A mistake in the address while transferring `DEFAULT_ADMIN_ROLE` is irreversible. If the role is granted to an address nobody controls and then revoked from the previous holder, control over the protocol is lost permanently, and with it any ability to fix anything.

An unbounded pause means that one unavailable key freezes the protocol indefinitely. Combined with M-02, where collateral is locked and interest keeps accruing during a pause, that turns from an inconvenience into direct harm to users.

Likelihood is low, since both scenarios require an operational mistake or a lost key rather than exploitation of the code.

**RECOMMENDATION**

Move the administrator role to `AccessControlDefaultAdminRules`. It requires the new holder to accept the role and introduces a mandatory delay, which makes a mistaken handover both visible and reversible.

Give the pause a maximum duration, after which it either lifts automatically or requires renewed confirmation. That bounds the consequences of an unavailable key and imposes some discipline on incident response along the way.

It is also worth spreading role holders across separate addresses. Today every role converges on one key, so losing it means losing control entirely rather than partially.

L-01

LOW

OPEN

## The Chainlink answer is not fully validated: the round id is ignored and there is no staleness headroom

### CATEGORY

Oracle layer

### LOCATION

src/core/ICFT/oracle/PriceOracle.sol#L298-L307

### DESCRIPTION

Reading a feed checks the sign of the answer and its age, but it does not unpack every returned field.

src/core/ICFT/oracle/PriceOracle.sol

solidity

```
301 (, int256 answer,, uint256 updatedAt,) = feed.latestRoundData();
302
303 if (answer <= 0 || updatedAt == 0) revert InvalidOracleAnswer();
304 if (block.timestamp > updatedAt + maxPriceAge) revert StaleOraclePrice();
```

The `answeredInRound` field is never read and never compared against `roundId`. The `updatedAt == 0` check does cover the case of an incomplete round, so the practical effect is smaller than in the classic version of this issue, but the completeness of the answer is formally never confirmed.

The `maxPriceAge` setting deserves a separate note. In the deployment it equals one hour, which matches the update period of the ETH/USD feed. There is no headroom, so an ordinary delay in publishing an update makes the price count as stale.

The oracle contains no sequencer uptime check for layer two networks. That is not critical within the current scope, but deploying to an L2 would add a window in which the price is formally fresh while the network has only just recovered.

### IMPACT

Once a price is treated as stale, every operation that needs a collateral valuation reverts, including repayment and liquidation. At that moment a borrower can neither reduce their debt nor be closed out, while interest keeps accruing. An error on the feed side therefore turns into a temporary block on exiting.

### RECOMMENDATION

Read `roundId` and `answeredInRound` and reject an answer whose round is not current. Set `maxPriceAge` with headroom over the feed update period, commonly one and a half to two periods, so that an ordinary publication delay does not cause a failure.

When moving to layer two networks, add a read of the sequencer uptime feed together with a grace period after it comes back online.

L-02

LOW

OPEN

## Risk parameter validation allows zero headroom between the target LTV and the liquidation threshold

### CATEGORY

Risk parameters

### LOCATION

`src/core/ICFT/risk/RiskEngine.sol#L233-L245`

### DESCRIPTION

The check that relates the target LTV to the liquidation threshold is not strict.

`src/core/ICFT/risk/RiskEngine.sol`

solidity

```
241  || targetLtvBps_ > liquidationThresholdBps_
```

Equal values are accepted. In that configuration a liquidation brings the position exactly back to the threshold, which means the health headroom after closing is zero.

This behaviour initially looked like a loop of repeated liquidations. Verification showed otherwise: once the position reaches the target it stays formally liquidatable because `calculateLTV` rounds down, it permits exactly one further call in which a single indivisible unit of collateral is seized thanks to the lower clamp on `seizeAmount`, and after that the LTV drops below the threshold and the process stops.

### IMPACT

The direct loss is negligible and amounts to one indivisible unit of collateral. What matters is different: in such a configuration the position sits exactly on the boundary after liquidation, and any minimal price movement makes it liquidatable again. The user pays the liquidator bonus repeatedly without ever gaining durable headroom.

### RECOMMENDATION

Make the check strict and require meaningful headroom between the target LTV and the liquidation threshold, on the order of three to five percentage points. Liquidation would then return a position to a state that survives small price swings.

`src/core/ICFT/risk/RiskEngine.sol`

solidity

```
-  || targetLtvBps_ > liquidationThresholdBps_  
+  || targetLtvBps_ + MIN_LTV_GAP_BPS > liquidationThresholdBps_
```

L-03

LOW

OPEN

## The project build is not reproducible: dependency paths conflict and the compiler version is not pinned

### CATEGORY

Build and dependencies

### LOCATION

```
remappings.txt  
foundry.toml  
package.json
```

### DESCRIPTION

The project does not build from the state of the repository. There are several reasons and they compound one another.

The `remappings.txt` file overrides the settings in `foundry.toml` and points `@openzeppelin/` and `@chainlink/` at `node_modules/`, where the installed versions use a different directory layout. Two imports therefore fail to resolve: `ReentrancyGuardUpgradeable` is absent from the expected path in the installed OpenZeppelin version, and `AggregatorV3Interface` in the installed Chainlink version does not live under `shared/interfaces` as the import states. The `lib/` directory holds yet another set of dependencies, partly incompatible with the first.

The build is separately broken by `script/CheckSepoliaState.s.sol`, where an emoji character sits inside an ordinary string literal, which the compiler treats as a parse error.

Every contract declares a floating version of `^0.8.20`, while compilation actually runs under `0.8.35`.

### IMPACT

Being unable to build the project from the repository hinders independent verification and reproduction of results. A floating compiler version means the bytecode depends on which version the build tool happens to select, which complicates checking that deployed bytecode matches the sources.

For this review the dependencies were vendored manually and script compilation was excluded. Those changes were temporary and are not part of the recommendation.

### RECOMMENDATION

Reduce the dependencies to a single source: either `lib/` through git submodules or `node_modules/`, but not both at once. Pin exact versions of OpenZeppelin and Chainlink that match the import paths actually used in the code. Remove the conflicting entries from `remappings.txt` and keep path resolution in `foundry.toml`.

Pin the compiler version exactly, for example `pragma solidity 0.8.35;`, and state the same version in the build settings. Add the `unicode` prefix to the string literal containing the emoji.

L-04

LOW

OPEN

## Collateral valuation makes an external call per asset while looping over the whole registry

### CATEGORY

Pool accounting

### LOCATION

src/core/ICFT/lending/LendingPool.sol#L685-L702  
src/core/ICFT/lending/LendingPool.sol#L762-L780

### DESCRIPTION

Computing the value of collateral iterates over the full list of supported assets and, for every non zero balance, calls into `RiskEngine`, which in turn calls the oracle.

src/core/ICFT/lending/LendingPool.sol

solidity

```
692 for (uint256 i = 0; i < assetsLength; ++i) {
693     address asset = supportedCollateralAssets[i];
694     uint256 balance = _getCollateralBalanceMemory(user, position, asset);
695
696     if (balance == 0) {
697         continue;
698     }
699
700     collateralValueUSD += riskEngine.getCollateralValueUSD(asset, balance);
701 }
```

The same pattern appears in `_hasAnyCollateral`. The `supportedCollateralAssets` array only ever grows through `setCollateralAsset`: a disabled asset stays in the list, because nothing removes entries from the array.

### IMPACT

The gas cost of `borrow`, `repay`, `withdrawCollateral` and liquidation grows linearly with the number of assets ever added, regardless of how many a given borrower actually uses and how many remain enabled. There are few assets today, so the effect is small. As the list grows, every operation becomes more expensive for all users, and with a long enough list liquidation could become unprofitable sooner than the economic model expects.

### RECOMMENDATION

Cap the length of the supported asset list with an explicit constant and check it when adding. Provide a way to remove an asset from the array rather than only clearing its `enabled` flag, so that disabled assets stop adding cost to every operation.

Alternatively, store the list of assets a position has actually deposited and iterate that instead of the global registry. The cost of an operation would then depend on the composition of the position rather than on the configuration history of the protocol.

L-05

LOW

OPEN

## Upgradeable contracts reserve no storage slots

### CATEGORY

Access control and upgradeability

### LOCATION

src/core/ICFT/lending/LendingPool.sol#L155-L179  
src/core/ICFT/oracle/PriceOracle.sol#L127-L137

### DESCRIPTION

Every protocol contract is upgradeable and runs behind a proxy. Their own state occupies sequential slots starting at zero. There is no reserved array of the `uint256[N] __gap` form in any of them.

The OpenZeppelin version five base contracts use namespaced storage, so no collision arises between them and the deriving contract. The risk is limited to the protocol's own state.

The code carries comments stating that the field order must not change, and the order is indeed respected. The project has already been through a reinitialisation that added the collateral asset registry, which means upgrades happen in practice.

src/core/ICFT/lending/LendingPool.sol

solidity

```
162 // Append-only storage layout:
163 // keep legacy state fields in their original order so already deployed
164 // proxies continue to read existing accounting values from the same slots.
```

### IMPACT

As long as new fields are appended strictly at the end, the layout stays compatible. That discipline rests on a comment and on developer attention rather than on a technical guarantee. Inserting a field in the middle, or introducing an intermediate contract into the inheritance chain, shifts the slots and causes an already deployed proxy to read unrelated values from the old addresses. For a contract that holds collateral and debt accounting, that means corrupted data.

### RECOMMENDATION

Add a reserved array at the end of every upgradeable contract and shrink it as new fields are introduced. Alternatively, move the protocol's own state onto namespaced storage, following the same scheme the OpenZeppelin base contracts use.

Whichever option is chosen, it is worth dumping the storage layout during the build and comparing it against the previous version automatically. That turns an agreement about field order into a check that does not depend on human attention.

L-06

LOW

OPEN

## Collateral is credited by the requested amount rather than the amount actually received

### CATEGORY

Pool accounting

### LOCATION

`src/core/ICFT/lending/LendingPool.sol#L250-L266`

### DESCRIPTION

When collateral is deposited in tokens, the position balance is increased by the requested amount immediately after the transfer, without checking how many tokens actually arrived at the contract.

`src/core/ICFT/lending/LendingPool.sol`

solidity

```
260 IERC20(asset).safeTransferFrom(msg.sender, address(this), amount);
261 erc20CollateralBalances[msg.sender][asset] += amount;
```

For ordinary ERC-20 tokens this is correct. For tokens that withhold a fee on transfer, and for tokens with a mutable balance, the credited amount will exceed what was actually received.

### IMPACT

The gap between credited and real collateral works in the user's favour and against the protocol: a loan is issued against collateral the contract does not hold. On withdrawal or liquidation the transfer is computed from the inflated recorded balance, and the shortfall surfaces for the last user whose funds cannot be paid out.

The risk is hypothetical today. The asset list is curated by the administrator and currently holds only ETH and ordinary tokens, and the specification places non standard token behaviour among the assumptions of the trust model. The finding is recorded as a caution for the case where the list is extended.

### RECOMMENDATION

Compute the credited amount as the difference between the contract balance before and after the transfer. That keeps the accounting correct regardless of how the token behaves, at the cost of one extra balance read.

`src/core/ICFT/lending/LendingPool.sol`

solidity

```
+ uint256 balanceBefore = IERC20(asset).balanceOf(address(this));
  IERC20(asset).safeTransferFrom(msg.sender, address(this), amount);
- erc20CollateralBalances[msg.sender][asset] += amount;
+ uint256 received = IERC20(asset).balanceOf(address(this)) - balanceBefore;
+ erc20CollateralBalances[msg.sender][asset] += received;
```

If support for such tokens is not planned, the restriction should be recorded in the documentation and checked when an asset is added to the registry.

L-07

LOW

OPEN

## The stepped rate curve jumps at segment boundaries and has no upper bound

### CATEGORY

Rate model

### LOCATION

src/core/ICFT/lending/InterestRateModel.sol#L136-L143  
src/core/ICFT/lending/InterestRateModel.sol#L162-L179

### DESCRIPTION

The rate is returned as one of four fixed values depending on the utilisation segment, with no interpolation inside a segment.

src/core/ICFT/lending/InterestRateModel.sol

solidity

```
136 function getBorrowRateBps(uint256 utilizationBps) external view returns (uint256) {  
137     RateConfig memory config = rateConfig;  
138  
139     if (utilizationBps < config.kink1Bps) return config.rate1Bps;  
140     if (utilizationBps < config.kink2Bps) return config.rate2Bps;  
141     if (utilizationBps < config.kink3Bps) return config.rate3Bps;  
142     return config.rate4Bps;  
143 }
```

The curve values match the MVP specification, so the question is not the numbers themselves but the shape: at a segment boundary the rate changes in a jump, for example from eight percent to fifteen when utilisation crosses the eighty percent mark.

Configuration validation checks the ordering of the kink points and that the rates are non decreasing, but it sets no upper bound. The `rate4Bps` value may be arbitrarily large.

### IMPACT

The jump at a boundary creates a noticeable difference in borrowing cost for a minimal change in utilisation, which gives a reason to time an operation around it. The effect is limited because accrual is tied to state changing moments and does not compound within a single block, so the gain from such timing is small.

The absence of an upper bound on the rate means a configuration change can sharply increase the cost of loans that are already open. That belongs to the powers of the role and is covered in the trust model, but a constraint at the code level would be more appropriate.

### RECOMMENDATION

Consider moving to a piecewise linear curve with interpolation inside each segment. Most lending protocols use that shape precisely because it removes the discontinuity at the boundary and makes borrowing cost predictable.

Set a sane upper bound on the annual rate in validation, so that a configuration change cannot make servicing a debt arbitrarily expensive.

I-01

INFORMATIONAL

OPEN

## Floating compiler version and incomplete documentation of the public interface

### CATEGORY

Code quality

### LOCATION

`src/core/**/*.sol`

### DESCRIPTION

All sixteen files declare a floating version of `pragma solidity ^0.8.20`, while the actual build runs under compiler `0.8.35`. The version is above `0.8.20`, so none of the issues of earlier releases apply here, but the dependence of the bytecode on the tool's choice remains.

The style check produced two hundred and fifty `NatSpec` issues: some public and external functions lack descriptions, parameter tags or return tags, or those tags do not match the signature. Thirty six issues relate to events whose key fields are not marked as indexed.

The tool separately flagged the size of `LendingPool`: eighteen state variables against a recommended limit of fifteen, and one function fifty five lines long.

### IMPACT

There is no direct security impact. Incomplete `NatSpec` makes independent verification and integrator work harder, and unindexed event fields complicate off chain observation of the protocol, which bears directly on incident response.

### RECOMMENDATION

Pin the compiler version exactly and align it with the build settings. Complete the `NatSpec` for the whole public interface, starting with the functions that move funds and set risk parameters. Mark addresses and identifiers in events as indexed wherever monitoring depends on them.

Large calculation functions are worth splitting, which makes both reading and future review easier.

I-02

INFORMATIONAL

OPEN

## The external whitepaper describes the protocol differently from the MVP specification and the implementation

### CATEGORY

Documentation

### LOCATION

`docs/MVP_SPEC.md`

### DESCRIPTION

Two documents were used during the review: the external protocol whitepaper and the `docs/MVP_SPEC.md` specification held in the repository. They diverge on a number of points, and in every such case the code follows the specification rather than the whitepaper.

The items below would look like implementation defects if measured against the whitepaper alone, but they are not.

Liquidation. The whitepaper describes it as open to any participant. In the code both entry points are gated by roles. The specification permits exactly this arrangement, stating that a restricted liquidation executor or bot is acceptable for the MVP, and separately noting that a full permissionless pipeline is not required at this stage.

The rate curve. The whitepaper lists segments of `5 / 7 / 10 / 15 / 25` percent. The specification defines `5 / 8 / 15 / 20`, and the implementation matches it exactly, including the kink points.

The utilisation metric. The specification prescribes the ratio of borrowed ICFT to the size of Fund A, which is what the code computes. The observation about the numerator is raised separately in M-01 and concerns how it is maintained, not the choice of metric.

The manual ICFT price. The specification describes it as an MVP arrangement with a later move to a market source. That does not remove the need for limiters, so H-01 still stands.

A protocol owned DEX, swap routing, governance mechanics and the final proxy design are all listed in the specification under work explicitly excluded from scope.

The Fund B reserve, vesting and a minimum borrow size are not mentioned in the specification at all. Those are whitepaper items and fall outside the current stage. To be clear, the question of covering losses from insolvency remains open regardless of what happens to Fund B and is raised in H-02, while a minimum borrow size has an independent economic justification and is raised in M-04.

### IMPACT

A divergence between documents has no effect on contract security by itself, but it does affect expectations. A reader of the whitepaper expects open liquidation, vesting and a reserve fund, whereas the deployed system behaves differently. For a review that creates a risk of misclassifying findings; for a user it creates a risk of poor decisions.

### RECOMMENDATION

Bring the external whitepaper in line with the actual MVP scope, or separate the target architecture from the current stage within it explicitly, stating what exists today and what is planned.

Before a mainnet launch, revisit the items deferred out of the MVP and make a decision on each: implement it or record the decision not to. That applies first of all to open liquidation, vesting and a mechanism for covering losses.

I-03

INFORMATIONAL

OPEN

## Interest compounds, and the accrual mode is not fixed in the specification

### CATEGORY

Rate model

### LOCATION

`src/core/ICFT/lending/LendingPool.sol#L546-L575`

### DESCRIPTION

Accrual runs through a global index. The increment is computed from the current value of the index rather than from its initial value.

`src/core/ICFT/lending/LendingPool.sol`

solidity

```
559 uint256 elapsed = block.timestamp - previousAccrualTime;
560 uint256 aprBps = interestRateModel.getBorrowRateBps(_calculateUtilization(totalBorrowedICFT));
561 uint256 interestFactor = (previousBorrowIndex * aprBps * elapsed) / (BPS * YEAR);
```

Because the multiplier is the index at the moment of accrual, every subsequent accrual is computed from an already increased base. This is compound interest with discrete capitalisation, and the capitalisation period is set not by a schedule but by how often the protocol is touched: the more frequently users transact, the more often accrual happens and the higher the effective annual rate.

It is worth noting separately that the repricing of elapsed periods is handled correctly here. Accrual is invoked before every state change, and utilisation does not move between two accruals, so an elapsed period is always charged at the rate that was in force during it.

The MVP specification says only that interest accrues in USD and is computed against the USD principal. It says nothing about simple or compound interest.

### IMPACT

A borrower pays somewhat more than the nominal annual rate implies, and the size of that difference depends on the activity of other users, which means it is not determined in advance. At a rate of twenty percent with frequent interaction the effective annual rate approaches twenty two percent.

This cannot be called an error, since the behaviour contradicts no stated requirement. The finding is recorded to capture the gap between the implementation and the common expectation of simple interest.

### RECOMMENDATION

State the chosen accrual mode in the specification explicitly. If compound interest was intended, say so and describe the rate as an effective annual rate, so that a user sees the real cost of borrowing.

If simple interest was intended, compute the increment from the initial value of the index rather than the current one, and reflect that in the model.

## APPENDIX A

# Invariants & Property Tests

The properties below were implemented as stateful Foundry invariants during the review and handed to the team for inclusion in continuous integration. The handler also moves prices, which is what separates this set from the one already in the repository.

| ID     | INVARIANT                                                                                              | METHOD               | RESULT                    |
|--------|--------------------------------------------------------------------------------------------------------|----------------------|---------------------------|
| INV-01 | The sum of all position debts equals the recorded aggregate debt, within one unit per position.        | Stateful fuzzing     | <b>HOLDS</b>              |
| INV-02 | The ETH balance of the contract is at least the sum of collateral across all positions.                | Stateful fuzzing     | <b>HOLDS</b>              |
| INV-03 | ICFT supply stays constant at one billion under any sequence of operations.                            | Stateful fuzzing     | <b>HOLDS</b>              |
| INV-04 | A user action cannot leave that user's position above the maximum LTV.                                 | Stateful fuzzing     | <b>HOLDS</b>              |
| INV-05 | A partial liquidation brings LTV to the target and never closes more than required.                    | Stateful fuzzing     | <b>HOLDS</b>              |
| INV-06 | Accrued interest is non decreasing and debt never falls without a repayment.                           | Stateful fuzzing     | <b>HOLDS</b>              |
| INV-07 | The ICFT balance of the contract covers the recorded protocol revenue.                                 | Stateful fuzzing     | <b>HOLDS</b>              |
| INV-08 | Fund A liquidity plus the tokens lent out equals the allocated size of the fund.                       | Stateful fuzzing     | <b>VIOLATED</b> →<br>M-01 |
| INV-09 | A position whose collateral is below its debt is closed in full, leaving no uncovered remainder.       | Stateful fuzzing     | <b>VIOLATED</b> →<br>H-02 |
| INV-10 | The LTV, interest accrual and partial liquidation formulas match the reference model on random inputs. | Differential testing | <b>HOLDS</b>              |

## APPENDIX B

# Automated Analysis Triage

Every static analysis hit was triaged by hand. No automated result became a finding on its own, and two informed the manual review.

| TOOL    | DETECTOR               | HITS | TRIAGE                                                                                                                                                             |
|---------|------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Slither | arbitrary-send-eth     | 1    | False positive as to exploitability: the function is role gated and checks the address, the amount and the result. The power itself is covered by the trust model. |
| Slither | weak-prng              | 1    | False positive: the modulo serves rounding up, not randomness.                                                                                                     |
| Slither | incorrect-equality     | 17   | Strict comparisons against internally computed values rather than an observable balance. No impact.                                                                |
| Slither | divide-before-multiply | 1    | Values are scaled to 1e18 and the division rounds up in the protocol's favour. Precision loss is on the order of one unit.                                         |
| Slither | unused-return          | 1    | Confirmed, and it informed <b>L-01</b> : part of the Chainlink answer is not checked.                                                                              |
| Slither | calls-loop             | 8    | Confirmed and raised as <b>L-04</b> : external calls inside a loop over the collateral asset list.                                                                 |
| Slither | timestamp              | 19   | Acceptable: time is used to measure the elapsed period and to check price staleness.                                                                               |
| solhint | use-natspec            | 250  | Confirmed and raised as <b>I-01</b> : the public interface is incompletely documented.                                                                             |
| solhint | gas-indexed-events     | 36   | Confirmed: key event fields are not indexed, which complicates monitoring.                                                                                         |

## APPENDIX C

# Disclaimer

---

This report is not, and must not be treated as, an endorsement or a rejection of any project or team. It offers no guarantee that the reviewed technology is free of errors, and it makes no assessment of its owners, its business model, or its commercial or legal compliance.

The review was performed against the source code at the commit listed in the Scope section. Any change to the code after that commit, and any deployment that does not match it byte for byte, falls outside the assurances given here. Users should confirm that the deployed bytecode corresponds to the reviewed source.

A security review is a time boxed exercise carried out with the greatest possible care. It reduces risk without eliminating it. The auditor recommends several independent reviews, a public bug bounty programme and continuous monitoring for any system that holds user funds.

The client may publish this report only in full and without modification.



# WhyUSayMyName

Independent smart contract security research

[github.com/WhyUSayMyName](https://github.com/WhyUSayMyName)

[x.com/WhyUSayMyName](https://x.com/WhyUSayMyName)